PRM393 — Lab 3: Cloud-Enabled Mobile App (Firebase, Google ARTEMIS, AI Code Review)

Oct 21, 2026 · @Lam Phuong

In this lab you build the app you designed in Lab 2 as a cloud-enabled Flutter product: users sign in, the app reports what they do and how it fails, it can be reconfigured without a new build, and its critical flows are checked by automated tests. You are graded on a working app **and** on the evidence that it works. The product (features, Firebase integration, architecture, UI) is worth 6 of 10 points; the engineering process (testing, code review, presentation slides and demo video, backed by a final report) is worth 4.

This handout states **what** must be true of your app, not which screens to build: those come from your Lab 2 design. Each requirement names its evidence; work that cannot be shown is graded as missing.

1. Lab information

Item	Detail
Course	PRM393 — Mobile Programming
Lab	Lab 3 — Cloud-Enabled Mobile App (Firebase, Google ARTEMIS, AI Code Review)
Work mode	Individual (unless the lecturer announces teams)
Prerequisite	Your approved Lab 2 deliverables: Figma file (pages 01–06), design/DESIGN.md, handoff/flutter-handoff.md
Required tools	Flutter (stable), Firebase project (Spark plan is enough), FlutterFire CLI, Google ARTEMIS (Python 3.12+, uv, ADB) with a multimodal model API key, GitHub, one AI code review tool
Target platform	Android device

Item	Detail
Submission	One GitHub repository link, one final report (PDF), one presentation slide deck (PDF), one demo video link

2. Learning outcomes

After this lab you can:

1. Implement a Figma design (flows, design tokens and components with states) in Flutter, so the built app matches its specification.
2. Structure a Flutter app in MVVM so that UI, state, business logic and external services can be changed and tested independently.
3. Integrate a cloud backend (Firebase) for identity, file storage, messaging and remote configuration, and handle each service's failure cases.
4. Design an analytics event plan that answers product questions, and instrument the app to record it.
5. Make crashes and handled errors observable in production.
6. Write a test plan, automate end-to-end tests of critical user flows with an AI testing agent, and judge whether its results can be trusted.
7. Use AI to review code, then judge each finding and act on it with a stated reason.

3. Task

Build the app you designed in Lab 2 (AI-Assisted UI/UX Design) as a working Flutter app on Firebase. Your Lab 2 deliverables are the specification: the persona and success criterion, the information architecture and the 3 user flows, the design system and components, and `handoff/flutter-handoff.md`. Your final app must:

- Implement **every screen** on the Lab 2 Final UI page (at least 8) and **every Lab 2 flow end to end**, including its alternative and error/recovery path, as they run in the Figma prototype.
- Build the **Lab 2 design system in code**: tokens as a `ThemeData` (`ColorScheme`, `TextTheme`, spacing and radius constants) and each Lab 2 component as one reusable widget with its states (section 5).

- Keep the **Lab 2 navigation structure** (top-level destinations and nesting). If the design has no sign-in or profile screen, add them with the same design system.
- Show the **loading, empty and error states** designed in Lab 2 on every screen that loads data.
- Use **real data** from the source named in the design: a public REST API, or Cloud Firestore when the app stores the user's own data (expenses, habits, deadlines). No hard-coded or mock data in the submitted build.
- Generate at least **1 file** (for example a PDF summary of what the user is viewing) and upload it to the cloud.
- Provide a **Firebase demo area**, usually on the Profile screen: notification center, file export and upload, Remote Config values, and the Crashlytics test actions.

Changes to the design. You may change the Lab 2 design when building it reveals a real problem (a platform constraint, an accessibility failure, an API limit). Record every change in `docs/design-traceability.md` with its reason, and update the Figma file so design and app stay in sync. An unrecorded difference between Figma and the app is graded as a defect.

Topic. Your approved Lab 2 topic. Changing topic needs the lecturer's approval by [date] and a revised Lab 2 design.

Tool note. Firebase consoles, ARTEMIS releases and AI review tools change over time. Pin the versions you use in `pubspec.yaml` and `README.md`. If a required service or tool is unavailable to you, tell the lecturer before starting; an equivalent may be approved, but the evidence requirements stay the same.

4. Required workflow

Follow the stages in this order: Plan, Architect, Integrate, Instrument, Test, Review, Report. Each stage produces a deliverable that the next stage depends on, and your Git history should show them in this order.

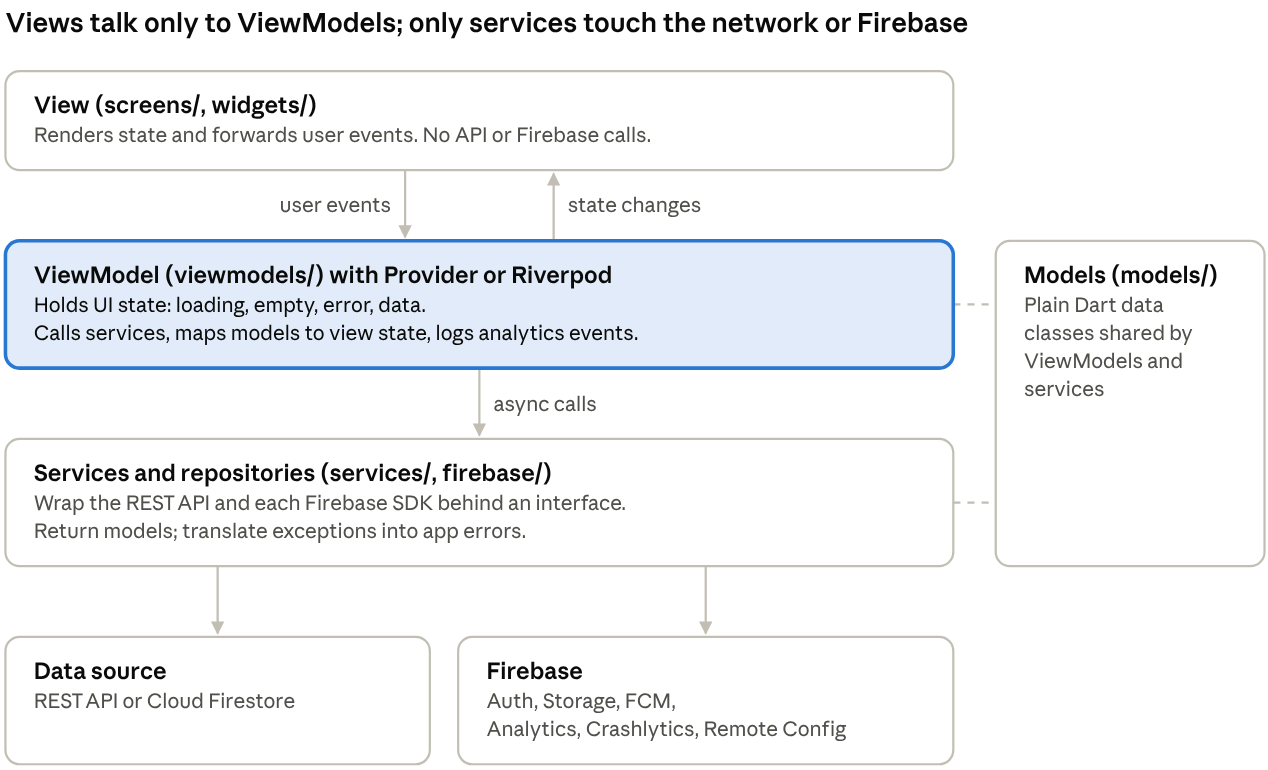
#	Stage	Output
1	Design traceability: map every Lab 2 screen, flow and component to a Flutter route, widget and data source	`docs/design-traceability.md`

#	Stage	Output
2	Architecture and theme: layers, state management, data flow; Lab 2 tokens → ThemeData, components → widgets	`docs/architecture.md`
3	Firebase setup and authentication	Working sign-in; `firebase/` config and security rules
4	Features: every Lab 2 flow with its states, then export and upload	Working screens with all states
5	Instrumentation: analytics event plan, Crashlytics, Remote Config, FCM	`docs/analytics-plan.md`; console screenshots
6	Test plan and ARTEMIS E2E tests	`docs/test-plan.md`; `e2e_test/`; test run evidence
7	AI code review and fixes	`docs/code-review.md`; fix commits
8	Final report, presentation slides and demo video	`report/Lab03_<StudentID>.pdf`, `slides/Lab03_<StudentID>_slides.pdf`; video link in `README.md`

References: Flutter: Guide to app architecture · Add Firebase to your Flutter app · google/artemis on GitHub

5. Architecture requirements

The app must follow MVVM with **Provider or Riverpod**. Dependencies point downwards only; a layer never imports from the layer above it.



MVVM layers · dependencies point down only

Rules that are checked in grading:

- No widget calls `http`, `FirebaseAuth.instance`, `FirebaseAnalytics.instance` or any other SDK directly. Every external call goes through a service.
- Business logic (filtering, ranking, aggregation, validation) lives in ViewModels or services, never in `build()` methods.
- Each data-loading screen has an explicit state model (for example a sealed class or enum for loading, empty, error, data) owned by its ViewModel.
- Services are injected (through Provider/Riverpod), so a ViewModel can be tested with a fake service.

`docs/architecture.md` must contain: the folder structure with one line per folder, a layer diagram (Mermaid is fine), one sequence diagram for a full flow (for example: search → API → ViewModel → chart), and a table of ViewModels with the services each one uses.

Suggested structure: `lib/` with `models/`, `services/`, `firebase/`, `viewmodels/`, `screens/`, `widgets/`, `utils/`. You may change it if `architecture.md` explains why.

Design system in code

The Lab 2 design system is part of the architecture, not decoration. Build it from the two mapping tables in your `flutter-handoff.md`:

- Tokens live in one place (`lib/theme/`): colours in a `ColorScheme`, text styles in a `TextTheme`, spacing, radius and elevation as named constants. Widgets read them from the theme; a hard-coded colour, font size or padding in a screen is a defect.
- Each Lab 2 component (button, text field, card, navigation, app bar, dialog, loading, empty and error state) is one reusable widget in `widgets/`, with its Lab 2 variants and states as parameters. Screens use these widgets, not one-off copies.
- The Lab 2 accessibility checklist must still hold in the built app: contrast, touch targets of at least 48 dp, body text of at least 14 sp, no meaning by colour alone, layout checked at 360 dp and 412 dp.

6. Firebase requirements

Integrate all six services below. For each one, the app must handle its failure case (no network, permission denied, user cancels) with a visible message rather than a crash. Each service is graded on behaviour **and** evidence.

Service	Minimum behaviour	Evidence
Authentication	Google Sign-In; app routes by auth state (signed out → sign-in screen, signed in → home); sign-out clears local user data; cancelled sign-in shows a message	Screenshots of both states; Users tab in the console
Cloud Storage	Upload the generated file under a per-user path (e.g. `reports/{uid}/…`); show progress and the download URL; security rules allow each user to read and write only their own folder	`storage.rules` in the repo; console file list

Service	Minimum behaviour	Evidence
Cloud Messaging (FCM)	Receive notifications in foreground, background and terminated states; ask for notification permission (Android 13+); keep a list of received notifications in the app; tapping one opens a relevant screen	Test message from the console received in each state
Analytics	Record the event plan below with parameters; set the user ID after sign-in	DebugView screenshots for every event
Crashlytics	Report one handled (non-fatal) error from a real error path and one test crash; attach user ID and at least one custom key	Crashlytics dashboard showing both, with keys
Remote Config	At least 2 parameters that change visible behaviour (e.g. max items in a ranking, a feature flag for a screen); in-app defaults; changing the value in the console changes the app without a new build	Before/after screenshots and console values

Analytics event plan

Before writing tracking code, design events that answer questions about how the app is used. Write the plan in `docs/analytics-plan.md` using the template in Appendix A. The plan must:

- Contain at least **7 events** that together cover: sign-in, sign-out, the main search or primary action, opening each kind of detail screen, and exporting/uploading the file.
- Use Firebase recommended event names where one fits (for example `login`, `search`), and `snake_case` custom names otherwise.
- Give each event the parameters needed to answer its question (for example the search term, the entity ID and name, the year), with types.
- State, for each event, the product question it answers (for example: "Which topics are searched most but exported least?").
- Contain no personal data in parameters (no e-mail, no name).

References: Firebase: Log events · Firebase: Get started with Crashlytics · Firebase: Remote Config for Flutter · Firebase: Secure files with Security Rules

7. Automated E2E testing with Google ARTEMIS

You must test, automatically, every flow a user depends on, using **Google ARTEMIS**: an open-source (Apache 2.0) AI agent that drives a real Android device or emulator from natural-language test instructions. Because the tester is itself an AI, you are graded on how well you make its results trustworthy, not only on how many tests pass.

Setup. Python 3.12+, `uv` and ADB (the ARTEMIS start script installs ADB, scrcpy and FFmpeg); an API key for a supported multimodal model (Gemini, Claude, GPT-4o or Qwen-VL). Use the **Flash** profile (about 3–5 s per step) for regression tests and the **Pro** profile only for the exploratory session. ARTEMIS is new and changes quickly: pin the commit you use in `README.md`.

`docs/test-plan.md` must contain (template in Appendix B):

- A list of the critical flows of your app, each mapped to the screens it uses.
- At least **10 test cases**, each with an ID, a precondition, the natural-language instruction, the expected result and the flow it covers.
- Coverage of: each Lab 2 user flow end to end, including its alternative and error paths; plus sign-in, the main search or primary action, each top-level tab, each kind of detail screen, export and upload, Remote Config values on screen, and sign-out.
- At least **1 error-path test** (for example: a search with no results shows the empty state, or no network shows the error state with a retry).
- At least **1 native interaction** (for example: the Google account picker, the notification permission dialog, or opening a push notification from the notification shade).
- **1 exploratory session** in the Pro profile (15–30 minutes, unscripted) and a list of the issues it found.

Rules for the tests:

- Each test is a `pytest` function that sends one instruction through the ARTEMIS Python SDK (`ArtemisClient`). The instruction states the starting point, the steps and exactly what must be visible at the end. "Check that the app works" is not a test.
- Assert on what the agent reports, not on `status == "SUCCESS"` alone: ask it to return the values it sees (result count, journal name, Remote Config value) as structured output and compare them with expected values.

- Run every test at least **3 times** and report the pass rate. A test that does not pass 3 of 3 is flaky: find out whether the cause is the app, timing or an ambiguous instruction, and record the fix.
- Add `Semantics` labels to key Flutter widgets (buttons, list items, charts). ARTEMIS reads the accessibility tree first and falls back to vision for custom Flutter UI, so labels make tests faster and more reliable, and the app more accessible.
- Tests do not depend on each other's order; each one starts from its stated precondition.
- Use a dedicated Google test account for sign-in, never a personal one. Keep its credentials and your model API key out of the repository (environment variables).
- At least one test also checks that its analytics event was logged (Analytics debug mode on; check the run's Logcat capture or DebugView).

Example shape of one test:

```
async def test_tc02_search_shows_results(artemis):
    result = await artemis.run(
        "Open the Journal Trend Analyzer app, signed in. On Home, search for
"
        "'machine learning'. Verify a publication list and a trend chart are
"
        "shown, and report the total publication count displayed."
    )
    assert result.status == "SUCCESS", result.error
    # then assert on the reported count, e.g. > 0
```

Place tests in `e2e_test/` with a `conftest.py` that creates the client (device serial, profile), grouped by feature (for example `test_auth.py`, `test_search.py`, `test_detail.py`, `test_export.py`, `test_profile.py`).

Evidence: the command used to run the suite; the trace ID, screenshots or replay of each run; a results table (test ID, runs, pass rate, average time); an explanation of every flaky, failed or skipped test; and the exploratory session report.

References: google/artemis on GitHub · AndroidWorld benchmark · Flutter: Accessibility

8. AI-assisted code review

You must use AI to review your code, and you must record how. AI finds; you decide.

1. **Review.** Run at least one AI review tool on your code before submission (for example GitHub Copilot code review, CodeRabbit, SonarQube/SonarCloud, Kodus). Review a pull request rather than the whole repository at once, so findings point to specific lines. Save the tool's output.
2. **Findings.** Record at least **5 findings** tied to a file and line, covering at least two kinds: bug, security, architecture (MVVM rule broken), performance, test gap, maintainability.
3. **Decision record.** For every finding, state whether you accepted, modified or rejected it, and why. The reason must refer to a requirement in this handout, a rule in section 5, a measurable effect (crash, latency, rebuild count) or a real constraint (scope, time, API limits). "The AI said so" and "I didn't like it" are not reasons.
4. **Fixes.** Fix at least **3** accepted findings, each in its own commit that names the finding ID. Link the commit in the record.
5. **Disclosure.** List every AI tool you used in this lab, including code generators and chat assistants, and for what purpose.

Keep all of this in `docs/code-review.md` (template in Appendix C).

9. Repository, final report, slides and demo video

Submit one GitHub repository named `PRM393_Lab03_<StudentID>` with this structure:

```
PRM393_Lab03_<StudentID>/
├── README.md
├── lib/                    (MVVM layers, section 5)
├── e2e_test/               (ARTEMIS pytest suite, section 7)
│   ├── conftest.py
│   ├── pyproject.toml
│   └── test_*.py
├── android/                (includes google-services.json)
├── firebase/
│   ├── storage.rules
│   └── remote-config-defaults.json
├── docs/
│   ├── design-traceability.md
│   ├── architecture.md
│   ├── analytics-plan.md
│   ├── test-plan.md
│   └── code-review.md
├── evidence/               (console, DebugView, Crashlytics, ARTEMIS traces
```

```
and screenshots)
├── report/
|   └── Lab03_<StudentID>.pdf
└── slides/
    └── Lab03_<StudentID>_slides.pdf
```

File	Purpose
`README.md`	Topic, API used, how to run the app and the tests, pinned versions, AI tools used, demo video link, and a map of where to find each deliverable
`docs/design-traceability.md`	Traceability matrix: each Lab 2 screen, flow and component → Flutter route, widget and data source, with status; every deviation from the design with its reason
`docs/architecture.md`	Section 5
`docs/analytics-plan.md`	Section 6 and Appendix A
`docs/test-plan.md`	Section 7 and Appendix B
`docs/code-review.md`	Section 8 and Appendix C

Do not commit service-account keys, API secrets or keystores. The Android `google-services.json` may be committed, because access is controlled by your security rules.

Final report (PDF, 8–12 pages). The complete written record of the lab; the slides and video summarise it. Include: project overview and Lab 2 recap (persona, flows, success criterion); design traceability and every deviation from the design with its reason; architecture and MVVM implementation; design system in code; Firebase design (one subsection per service, with its failure handling); analytics event plan and DebugView evidence; Crashlytics and Remote Config evidence; ARTEMIS test plan, results and the exploratory session; AI code review findings and decisions; challenges encountered; lessons learned; and a list of AI tools used. Every claim that something works carries a screenshot or a link to evidence in the repository.

Presentation slides (12–15 slides, PDF). A viewer who has not seen the code should understand what you built and how you know it works. Include at least: topic, persona and the Lab 2 success criterion; key screens side by side, Figma vs app, with the deviations; architecture (MVVM layer diagram); design system in code (tokens → theme, components

→ widgets); Firebase design, one slide per service with its failure handling; analytics events with DebugView evidence; Crashlytics and Remote Config evidence; ARTEMIS test plan and pass-rate results; code review findings and decisions; challenges and lessons learned. Link to the `docs/` files instead of copying them, and list any AI tool used to make the slides. The slides are also used in the verification review (section 11).

Demo video (5–10 minutes). Walk through each flow in the order of your test plan: sign-in, each Lab 2 flow including its error path, export and upload, a push notification received, a Remote Config change applied without a rebuild, the Crashlytics demo, an ARTEMIS test run on the device, and one code review finding with its fix. Show the Firebase console where it proves the result.

10. Grading rubric (10 points)

Each sub-criterion is graded as full marks, partial marks or zero. Partial marks are given when the requirement is met in part, for example on some screens, services or flows but not all. Product criteria total 6 points (functional 2.5, Firebase 2, architecture 0.75, UI/UX 0.75); process criteria total 4 (testing 1.5, code review 0.5, presentation slides and video 2). The final report is required but not scored on its own: it carries the evidence for the other criteria, so a criterion without evidence in the report or repository is graded as not met.

Criterion	Pts	Full marks when	Points are lost when
Functional requirements	**2.5**		
Flows and navigation	1.0	Every Lab 2 screen (≥ 8) and all 3 Lab 2 flows work end to end, incl. the error path; sign-in and profile added; Back works everywhere	Lab 2 screens or flows missing; dead ends; unreachable screens
Data and content	0.75	Real data from the source named in the design (API or Firestore); each screen shows the content in screen-spec.md; generated file reflects what the user sees	Hard-coded or mock data; screens show different content from the spec

Criterion	Pts	Full marks when	Points are lost when
States and error handling	0.75	Loading, empty and error states with retry on every data screen; API and Firebase failures show a message	Spinner forever; blank screen on error; app crashes on no network
Firebase integration and analytics	**2.0**		
Authentication	0.25	Sign-in, auth-state routing, sign-out, cancel handled	Signed-out user reaches app screens; no sign-out
Cloud Storage	0.25	Per-user path, progress, URL shown, rules restrict access	Open rules (`allow read, write: if true`); no feedback
Cloud Messaging	0.5	Received in all three app states; permission requested; list in app; tap navigates	Only foreground; no permission handling
Analytics	0.5	Event plan mapped to Lab 2 flows, with questions and parameters; every event visible in DebugView; user ID set	Events without parameters; plan missing; personal data in parameters
Crashlytics	0.25	Non-fatal from a real error path and test crash both visible, with user ID and custom key	Only a forced crash; no evidence
Remote Config	0.25	≥ 2 parameters change behaviour; defaults in app; change applied without rebuild	Values only printed, not used; no defaults
Architecture (MVVM + Provider/Riverpod)	**0.75**		
Layer separation	0.5	All rules in section 5 hold; `architecture.md` matches the code	SDK calls or business logic in widgets

Criterion	Pts	Full marks when	Points are lost when
State management	0.25	Explicit state model per screen; services injected; no global mutable state	`setState` for shared state; singletons accessed from widgets
UI/UX and application quality	**0.75**		
Fidelity to the Lab 2 design system	0.5	Screens match the Lab 2 Final UI; tokens come from the theme and components from the Lab 2 library; every deviation recorded with a reason	Hard-coded colours or sizes; one-off widgets instead of Lab 2 components; unrecorded deviations
Usability and accessibility	0.25	Lab 2 accessibility checklist holds in the app (contrast, ⩾ 48 dp targets, ⩾ 14 sp text); works at 360 and 412 dp; feedback for every action; no overflow errors	Overflow stripes; tiny targets; no feedback on upload or export
Automated E2E testing (Google ARTEMIS)	**1.5**		
Test plan and coverage	0.5	⩾ 10 cases covering every item in section 7, incl. an error path, a native interaction and an exploratory session	Fewer cases; happy paths only; plan missing
Test quality	0.5	Instructions state verifiable expected results; assertions on reported values, not status alone; ⩾ 3 runs per test; Semantics labels on key widgets	Vague instructions; only the SUCCESS status checked; single runs; order-dependent tests
Execution evidence	0.5	Traces or screenshots for every run; results table with pass rates; flaky and failed tests explained	No run evidence; results claimed without logs
AI-assisted code review	**0.5**		

Criterion	Pts	Full marks when	Points are lost when
Findings and decisions	0.25	⩾ 5 findings tied to lines, each accepted/modified/rejected with a valid reason; tools disclosed	Fewer findings; reasons missing or "the AI said so"
Fixes	0.25	⩾ 3 fixes, each in a commit linked to its finding	Fixes not traceable; no commits
Presentation slides and demo video	**2.0**		
Presentation slides	1.0	Every slide in section 9 present; each "it works" claim backed by a screenshot from the app, a console or a test run; one message per slide, readable at a glance	Missing topics; walls of text; claims without evidence; screenshots without explanation
Demo video	1.0	Every item in section 9 shown live on a device in 5–10 minutes; Firebase console shown where it proves the result; clear narration	Items missing; edited-in results instead of live runs; video unavailable or too long

Access rule. If the repository, report, slides or video link cannot be opened at the deadline, the criteria that depend on it are graded 0 until it is fixed, and the late policy applies.

11. Academic integrity, verification and submission

Using AI is allowed and, for the code review, required. What is not allowed is submitting code you cannot explain. To make the work checkable:

- Your Git history must show work over time, not one upload at the end (at least [N] commits over at least [N] days), following the stages in section 4.
- Firebase evidence must come from your own project: the project ID in console screenshots must match the one in your `google-services.json`.

- Each student may be asked for a short review of about 5 minutes: run one ARTEMIS test, change one Remote Config value live, and explain one ViewModel and one code review decision. Marks can be adjusted after this review.
- Undisclosed AI use, fabricated evidence or test results, or copying another student's work are handled under the university's academic integrity policy.

Submission checklist

- [ ] Topic and API approved by the lecturer
- [ ] `design-traceability.md` and `architecture.md` written before feature work
- [ ] MVVM and design-system rules in section 5 hold across the codebase; every Lab 2 screen and flow traced
- [ ] All six Firebase services work, each with its failure case handled
- [ ] Storage security rules restrict each user to their own folder
- [ ] Analytics event plan written; every event seen in DebugView
- [ ] Crashlytics shows a non-fatal error and a test crash with keys
- [ ] Two Remote Config values change the app without a rebuild
- [ ] Test plan with ⩾ 10 cases, incl. an error path, a native interaction and an exploratory session
- [ ] Full ARTEMIS run (⩾ 3 runs per test) recorded with traces and a pass-rate table
- [ ] ⩾ 5 review findings with decisions; ⩾ 3 fixes in linked commits
- [ ] No secrets committed
- [ ] README lists versions, AI tools used and the video link
- [ ] Repository, report, slides and video open in a private browser window

Appendix A — Analytics event plan template

Event	Trigger	Parameters (type)	Question it answers
`login`	Sign-in succeeds	`method` (string)	How many users return each week?
`search`	User runs a search	`search_term` (string), `result_count` (int)	Which topics are searched, and which return nothing?

Event	Trigger	Parameters (type)	Question it answers
`view_<entity>`	A detail screen opens	`<entity>_id` (string), `<entity>_name` (string)	Which items are opened most after a search?
`export_report`	File generated and uploaded	`topic` (string), `file_size_kb` (int)	Which searches lead to an export?
`logout`	User signs out	—	How long do sessions last before sign-out?

Map each event to the Lab 2 flow step it measures, so the analytics show where users leave each flow before reaching its goal, and how close the app comes to your Lab 2 success criterion.

Appendix B — Test case template

ID	Flow	Precondition	Steps	Expected result	Type	Result
TC-01	Sign-in	App installed, signed out	Launch → tap Google sign-in → pick test account	Home screen shows; bottom bar visible	Native	Pass rate, e.g. 3/3
TC-02	Search	Signed in	Enter topic → search	Results list and chart show; count > 0	Happy path	
TC-03	Search	Signed in	Enter a term with no results → search	Empty state with message shows	Error path	

Appendix C — Code review record template

ID	Tool	File:line	Kind	Finding (summary)	Decision	Reason	Fix commit
CR-01	CodeRabbit	`lib/screens/home_screen.dart:88`	Architecture	Widget calls the API client directly	Accepted	Breaks the section 5 rule; blocks testing with a fake service	`a1b2c3d`
CR-02	SonarCloud	`lib/services/report_service.dart:41`	Maintainability	Method is 120 lines long	Modified	Split into build and upload; kept the PDF layout in one place	`d4e5f6a`

ID	Tool	File:line	Kind	Finding (summary)	Decision	Reason	Fix commit
CR-03	Copilot	`lib/viewmodels/keyword_vm.dart:23`	Performance	Suggests caching all API responses	Rejected	Data must be live for the trend view; a 5-minute cache would hide new results	—